

河北科技大學

Course Design Report

Student Name: MATEEN MUHAMMAD

Student ID: P230702040

Major : Computer Science and Technology

Class: class of 2023

Course Name: Data Structure Course Design

Academic Year and Semester : 2025-2026-2 =-3

Supervising teacher: 郑 广

December 2025

Data structure course design performance evaluation form

Student Name	MATEEN MUHAMMAD	Student ID	P230702040	Results	
class	Class of 2023	Start ~End time	2025/12/08~2025/12/12		
Design topic	The Address Bock Management				
Comments	Supervising teacher：_____ Year Month Day				

Table of Contents

1.Design Assignment Sheet 1 1

2. Group Project and My Task 3

3.Brief Introduction to Program Function 3

4. Main Content 4

4.1 Design Analysis 4

4.2 Program Structure 4

4.3 Functions of Each Module and Program Description 5

4.4 Source Program 6

4.5Operation Method 17

4.6 Test results 20

4.7Design Experience 22

1. Design Assignment Sheet

1.1 Main Tasks and Objectives

The main objective of this course design is to develop an Address Book Management System using the C programming language.

The system is based on a doubly linked list, which is used to store and manage contact information dynamically.

Through this project, students are expected to understand the principles of linked list operations, pointer manipulation, and basic file input/output techniques.

The design also aims to improve problem-solving skills and practical programming experience.

1.2 Main Content and Basic Requirements

(1) The address book data is stored using a doubly linked list structure.

Each node contains personal information including:

name, street, city, postcode, email, country, and state.

(2) The system should provide the following basic functions:

- Add new contact information
- Display all contact records
- Search for a contact by name
- Delete a contact by name
- Save the address book data to a file

- Load the address book data from a file

(3) During program execution, clear messages and results should be displayed on the screen to ensure easy user interaction.

(4) The program should have a clear structure, reasonable module division, and readable source code.

1.3 Development Tools

- Programming Language: C
- Development Environment: Dev-C++
- Operating System: Windows

1.4 Project Schedule

December 4:

Analyze the course design requirements and determine the system functions.

Select the doubly linked list as the main data structure.

December 5:

Design the data structure for address book records and implement the contact input and insertion functions.

December 6:

Implement search, display, and delete functions for the linked list.

December 7:

Add file saving and loading functions to store and restore address book

data.

December 8:

Debug and test the program using different data cases, verify correctness, and complete the course design report.

2.Group Project and My Task

This project was completed as an individual assignment.

My responsibilities included designing the data structure, implementing all program functions, testing the system, debugging errors, and writing the course design report.

3

Data structure course design

Through this work, I gained hands-on experience with dynamic memory allocation, linked list operations, and file handling in C.

3. Brief Introduction to Program Functions

The address book management system provides the following main functions:

- Add new contact information
- Display all contact records

- Search for a contact by name
- Delete a contact by name
- Save address book data to a file
- Load address book data from a file

These functions allow users to manage personal contact information efficiently.

4. Main Content

4.1 Design Analysis

The address book system is based on a doubly linked list, which allows flexible insertion and deletion of records.

Compared with arrays, linked lists do not require continuous memory and are suitable for dynamic data storage.

Each contact is stored as a node containing personal information and pointers to the previous and next nodes.
This design improves efficiency and simplifies record management.

4.2 Program Structure

The program is divided into several modules:

- Input module
- Linked list operation module
- File operation module
- Output and display module

Each module performs a specific task, making the program structure clear and easy to maintain.

4.3 Functions Of Each Module and Program Description

- enter()

Adds a new contact to the address book by allocating memory and inserting a node into the linked list.

- display()

Traverses the linked list and displays all stored contact information.

- search()

Searches for a specific contact by name and displays the result.

- delete()

Deletes a contact record based on the given name and updates linked list pointers.

- save()

Saves all contact records to a text file.

- load()

Loads contact records from a text file and rebuilds the linked list.

4.4 Source Program

This section presents the main source code of the Address Book Management System.

The program is written in C and uses a doubly linked list to store contact information dynamically.

Structure Definition

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
struct Address {
char name[50];
char street[50];
char city[50];
char postcode[20];
char email[50];
char country[30];
char state[30];
struct Address* prev;
struct Address* next;
};
```

Function to Add a New Contact

```
struct Address* enter(struct Address* head) {
```

```
struct Address* newNode = (struct Address*)malloc(sizeof(struct
Address));
printf("Enter Name: ");
scanf("%s"
, newNode->name);
printf("Enter Street: ");
scanf("%s"
, newNode->street);
printf("Enter City: ");
scanf("%s"
, newNode->city);
printf("Enter Postcode: ");
scanf("%s"
, newNode->postcode);
printf("Enter Email: ");
scanf("%s"
, newNode->email);
printf("Enter Country: ");
scanf("%s"
, newNode->country);
printf("Enter State: ");
scanf("%s"
```

```
, newNode->state);
newNode->prev = NULL;
newNode->next = head;
if (head != NULL)
head->prev = newNode;
return newNode;
}
```

Function to Search a Contact by Name

```
void search(struct Address* head, char key[]) {
while (head != NULL) {
if (strcmp(head->name, key) == 0) {
printf("Record found:\n");
printf("Name: %s\nStreet: %s\nCity: %s\nPostcode:
%s\nEmail: %s\nCountry: %s\nState: %s\n"
,
head->name, head->street, head->city,
head->postcode, head->email,
head->country, head->state);
return;
}
head = head->next;
}
```

```
printf("Record not found.\n");
}
Function to Search a Contact by Name
void search(struct Address* head, char key[]) {
while (head != NULL) {
if (strcmp(head->name, key) == 0) {
printf("Record found:\n");

printf("Name: %s\nStreet: %s\nCity: %s\nPostcode:
%s\nEmail: %s\nCountry: %s\nState: %s\n"
,
head->name, head->street, head->city,
head->postcode, head->email,
head->country, head->state);
return;
}
head = head->next;
}
printf("Record not found.\n");
}
Function to Delete a Contact
struct Address* deleteRecord(struct Address* head, char key[]) {
```

```
struct Address* current = head;
while (current != NULL) {
if (strcmp(current->name, key) == 0) {
if (current->prev != NULL)
current->prev->next = current->next;
else
head = current->next;
if (current->next != NULL)
current->next->prev = current->prev;
free(current);
printf("Record deleted successfully.\n");
return head;
}
current = current->next;
}
printf("Record not found.\n");
return head;
}
```

Function to Save Data to File

```
void saveToFile(struct Address* head) {
FILE* fp = fopen("address
book.txt"
```

```
,

_
if (fp == NULL) {
printf("Cannot open file.\n");
return;
"w");
}
while (head != NULL) {
fprintf(fp,
"%s %s %s %s %s %s %s\n"
,
head->name, head->street, head->city,
head->postcode, head->email,
head->country, head->state);
head = head->next;
}
fclose(fp);
}
```

Function to Load Data from File

```
struct Address* loadFromFile() {
FILE* fp = fopen("address
_
```

```
if (fp == NULL) return NULL;
book.txt"
,
"r");
struct Address* head = NULL;
struct Address* tail = NULL;
while (1) {
struct Address* newNode = (struct Address*)malloc(sizeof(struct
Address));
if (fscanf(fp,
"%s %s %s %s %s %s %s",
newNode->name, newNode->street,
newNode->city, newNode->postcode,
newNode->email, newNode->country,
newNode->state) != 7) {
free(newNode);
break;
}
newNode->next = NULL;
newNode->prev = tail;
if (tail != NULL)
tail->next = newNode;
```

```
else
head = newNode;
tail = newNode;
}
fclose(fp);
return head;
}
```

Main Function

```
int main() {
struct Address* head = NULL;
int choice;
char name[50];
do {
printf("\n1. Enter\n2. Display\n3. Search\n4. Delete\n5. Save\n6.
Load\n0. Exit\n");
scanf("%d"
, &choice);
switch (choice) {
case 1: head = enter(head); break;
case 2: display(head); break;
case 3:
printf("Enter name to search: ");
```

scanf("%s"

, name);

search(head, name);

break;

case 4:

printf("Enter name to delete: ");

scanf("%s"

, name);

head = deleteRecord(head, name);

break;

case 5: saveToFile(head); break;

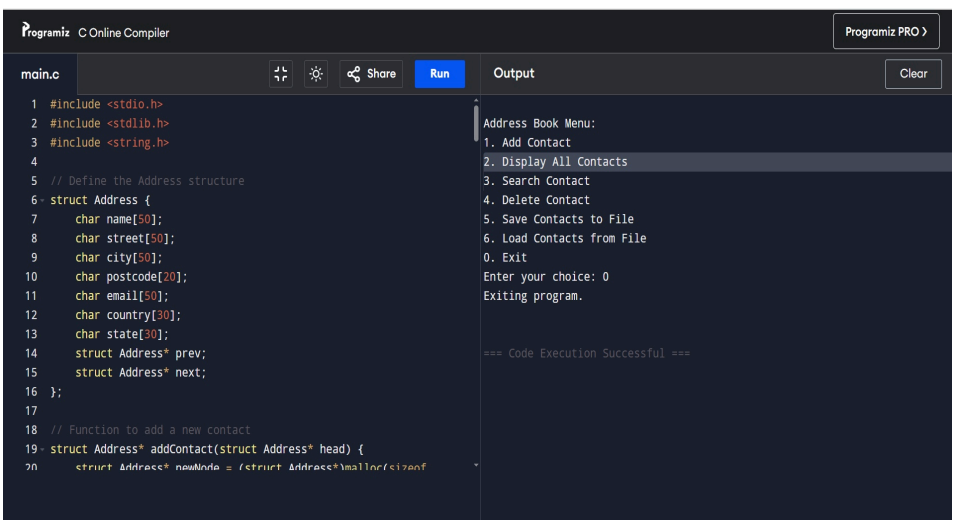
case 6: head = loadFromFile(); break;

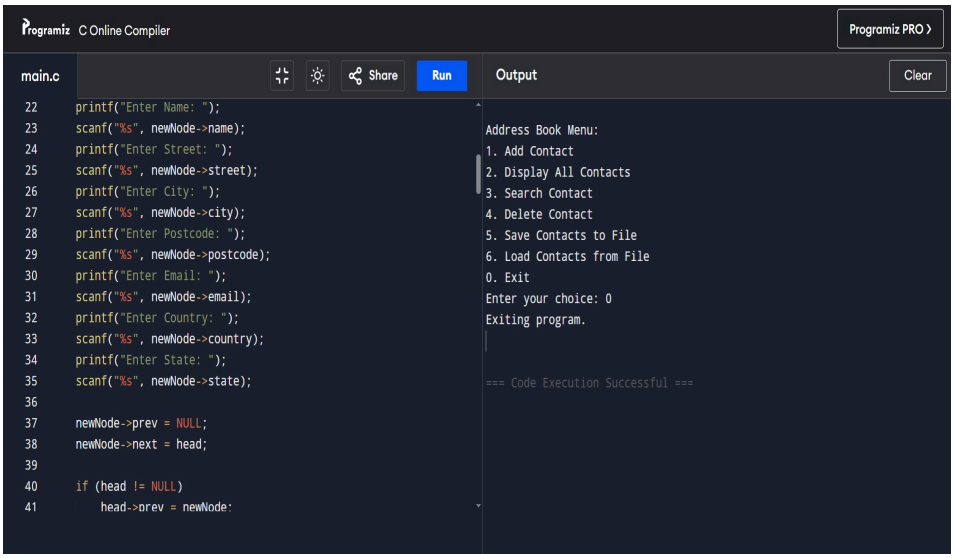
}

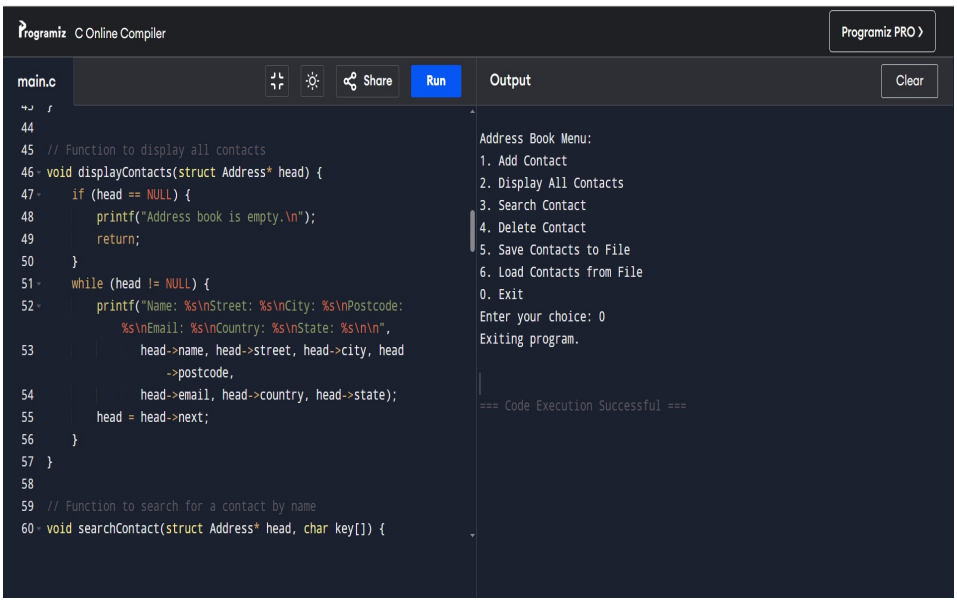
} while (choice != 0);

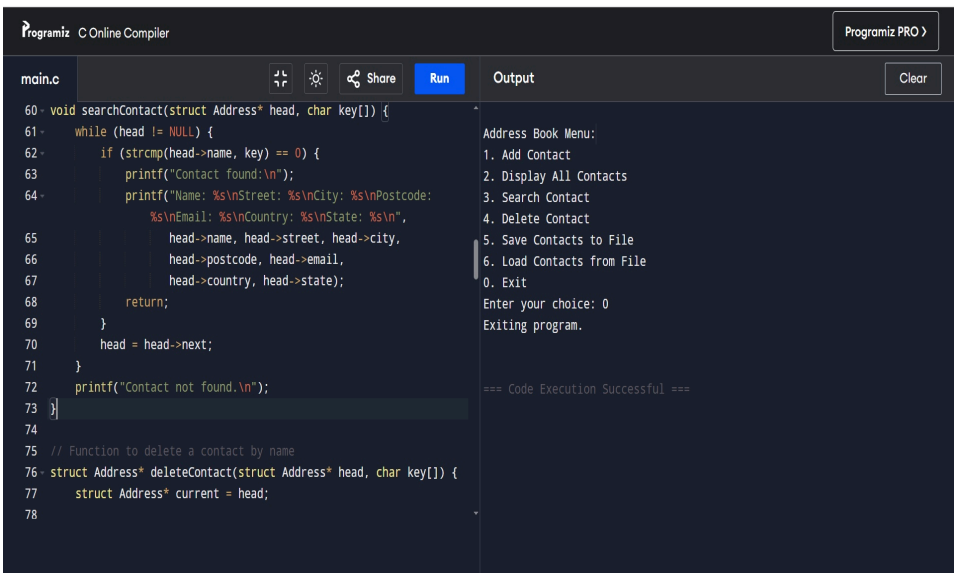
return 0;

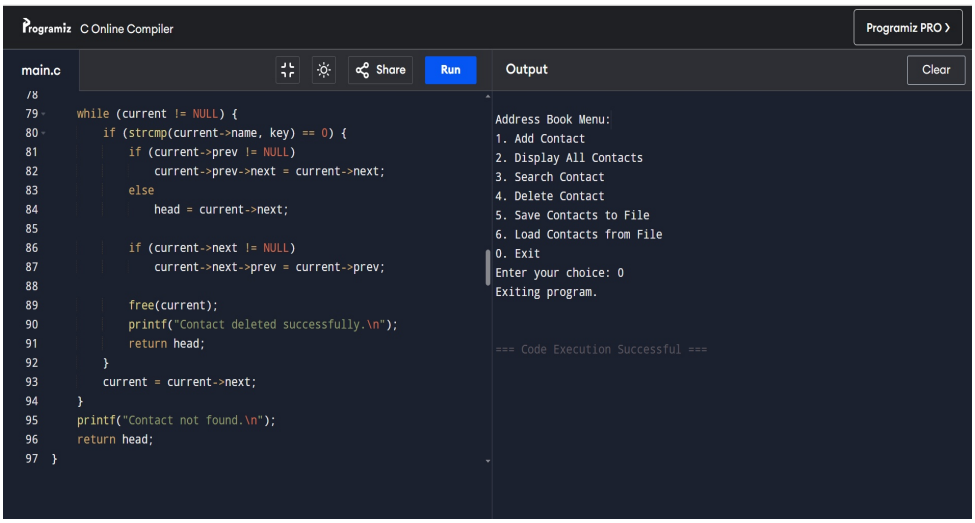
}

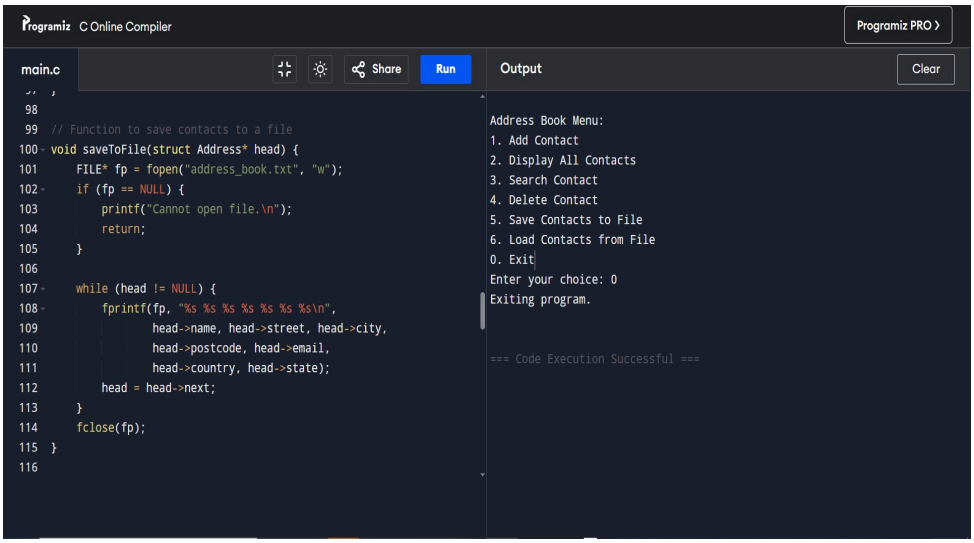


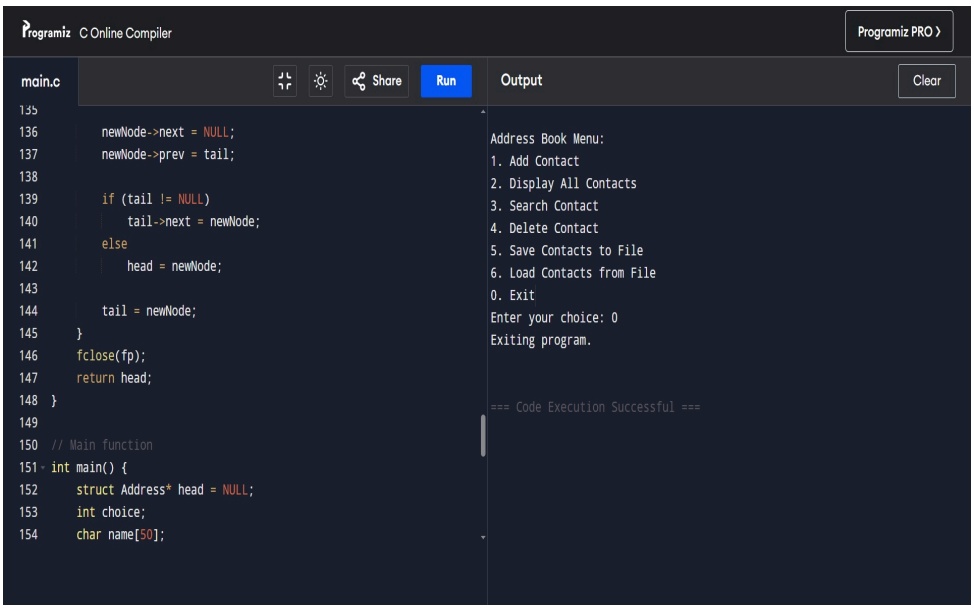

Programiz C Online Compiler
Programiz PRO
main.c
Share
Run
Output
Clear
printf("Enter Name: ");
scanf("%s", newNode->name);
printf("Enter Street: ");
scanf("%s", newNode->street);
printf("Enter City: ");
scanf("%s", newNode->city);
printf("Enter Postcode: ");
scanf("%s", newNode->postcode);
printf("Enter Email: ");
scanf("%s", newNode->email);
printf("Enter Country: ");
scanf("%s", newNode->country);
printf("Enter State: ");
scanf("%s", newNode->state);
newNode->prev = NULL;
newNode->next = head;
if (head != NULL)
head->prev = newNode;
Address Book Menu:
1. Add Contact
2. Display All Contacts
3. Search Contact
4. Delete Contact
5. Save Contacts to File
6. Load Contacts from File
0. Exit
Enter your choice: 0
Exiting program.
=== Code Execution Successful ===

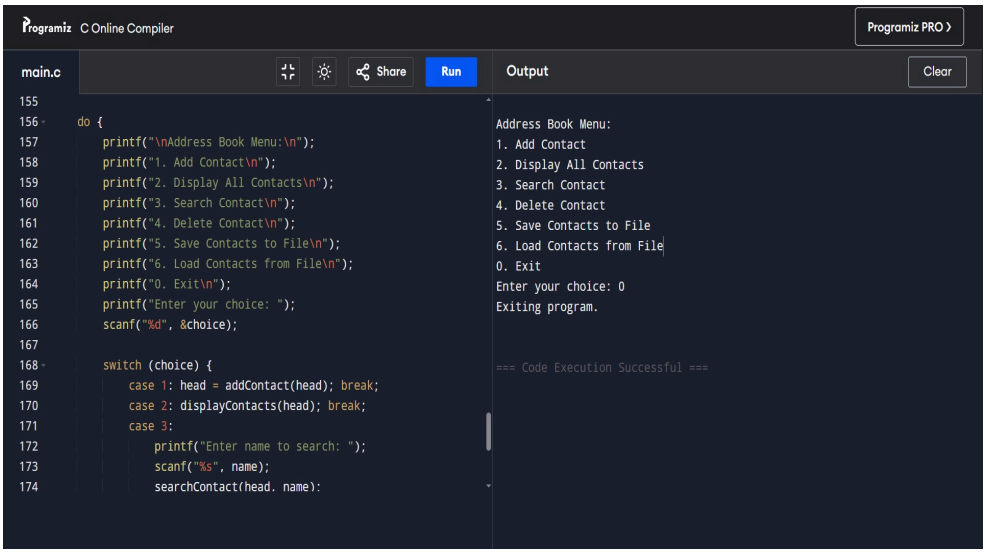

Programiz C Online Compiler
Programiz PRO
main.c
Share
Run
Output
Clear
// Function to display all contacts
void displayContacts(struct Address* head) {
if (head == NULL) {
printf("Address book is empty.\n");
return;
}
while (head != NULL) {
printf("Name: %s\nStreet: %s\nCity: %s\nPostcode:
%s\nEmail: %s\nCountry: %s\nState: %s\n\n",
head->name, head->street, head->city, head
->postcode,
head->email, head->country, head->state);
head = head->next;
}
}
// Function to search for a contact by name
void searchContact(struct Address* head, char key[]) {
Address Book Menu:
1. Add Contact
2. Display All Contacts
3. Search Contact
4. Delete Contact
5. Save Contacts to File
6. Load Contacts from File
0. Exit
Enter your choice: 0
Exiting program.
=== Code Execution Successful ===

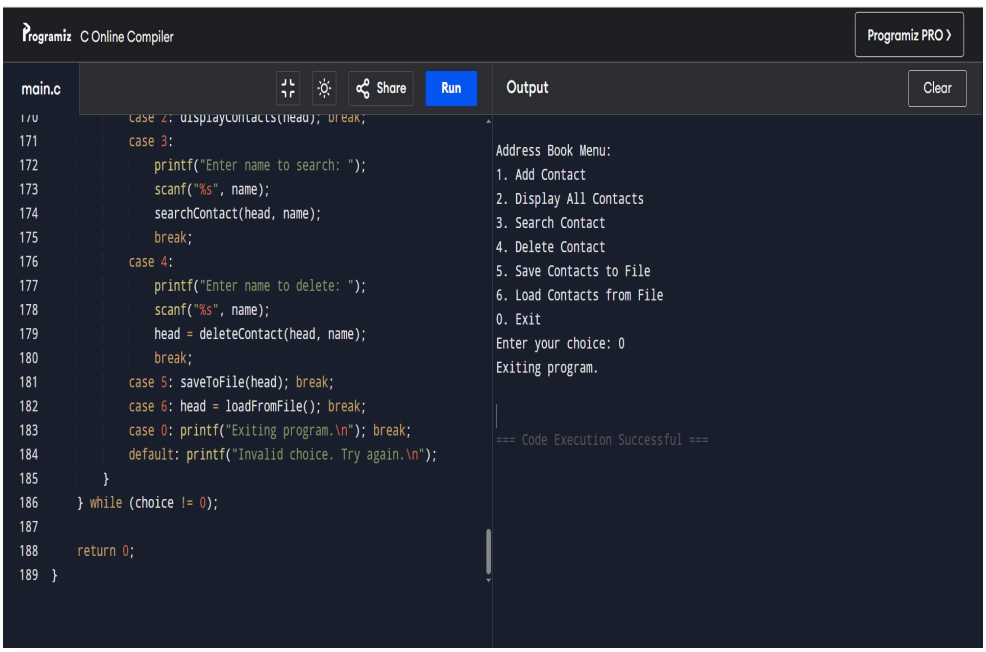

Programiz C Online Compiler
Programiz PRO
main.c
Share
Run
Output
Clear
void searchContact(struct Address* head, char key[]) {
while (head != NULL) {
if (strcmp(head->name, key) == 0) {
printf("Contact found:\n");
printf("Name: %s\nStreet: %s\nCity: %s\nPostcode:
%s\nEmail: %s\nCountry: %s\nState: %s\n",
head->name, head->street, head->city,
head->postcode, head->email,
head->country, head->state);
return;
}
head = head->next;
}
printf("Contact not found.\n");
}
// Function to delete a contact by name
struct Address* deleteContact(struct Address* head, char key[]) {
struct Address* current = head;
Address Book Menu:
1. Add Contact
2. Display All Contacts
3. Search Contact
4. Delete Contact
5. Save Contacts to File
6. Load Contacts from File
0. Exit
Enter your choice: 0
Exiting program.
=== Code Execution Successful ===


Programiz C Online Compiler
Programiz PRO
main.c
Share
Run
Output
Clear
while (current != NULL) {
if (strcmp(current->name, key) == 0) {
if (current->prev != NULL)
current->prev->next = current->next;
else
head = current->next;
if (current->next != NULL)
current->next->prev = current->prev;
free(current);
printf("Contact deleted successfully.\n");
return head;
}
current = current->next;
}
printf("Contact not found.\n");
return head;
}
Address Book Menu:
1. Add Contact
2. Display All Contacts
3. Search Contact
4. Delete Contact
5. Save Contacts to File
6. Load Contacts from File
0. Exit
Enter your choice: 0
Exiting program.
=== Code Execution Successful ===


Programiz C Online Compiler
Programiz PRO
main.c
Share
Run
Output
Clear
// Function to save contacts to a file
void saveToFile(struct Address* head) {
FILE* fp = fopen("address_book.txt", "w");
if (fp == NULL) {
printf("Cannot open file.\n");
return;
}
while (head != NULL) {
fprintf(fp, "%s %s %s %s %s %s %s\n",
head->name, head->street, head->city,
head->postcode, head->email,
head->country, head->state);
head = head->next;
}
fclose(fp);
}
Address Book Menu:
1. Add Contact
2. Display All Contacts
3. Search Contact
4. Delete Contact
5. Save Contacts to File
6. Load Contacts from File
0. Exit
Enter your choice: 0
Exiting program.
=== Code Execution Successful ===


Programiz C Online Compiler
Programiz PRO
main.c
Share
Run
Output
Clear
newNode->next = NULL;
newNode->prev = tail;
if (tail != NULL)
tail->next = newNode;
else
head = newNode;
tail = newNode;
}
fclose(fp);
return head;
}
// Main function
int main() {
struct Address* head = NULL;
int choice;
char name[50];
Address Book Menu:
1. Add Contact
2. Display All Contacts
3. Search Contact
4. Delete Contact
5. Save Contacts to File
6. Load Contacts from File
0. Exit
Enter your choice: 0
Exiting program.
=== Code Execution Successful ===





4.5 Operation Method

The following steps explain how to use the address book management system:

(1) Starting the Program

Run the program to display the main menu on the screen.

Each option is clearly numbered, and you can choose an action by entering the corresponding number .

(2) Entering New Information

Select the option for adding a new contact.

The program will ask you to enter details such as the name, street, city, postcode, email, country, and state.

After filling in all fields, the new record will be added to the address book.

(3) Searching for a Record

Choose the Search option if you want to find someone in the address book.

Simply type the person’s name, and if the record exists, the program will display all related details.

(4) Deleting a Record

To remove a contact, select the Delete option.

You will be asked to enter the name of the person you want to delete.

If the name is found, the record will be removed from the list.

(5) Saving and Loading Data

You can save your address book at any time using the Save option.

All records will be written to a text file.

Later , you can use the Load option to restore the saved data back into the program.

(6) Exiting the Program

To close the system, choose option 0 from the main menu.

The program will finish and return to the console.

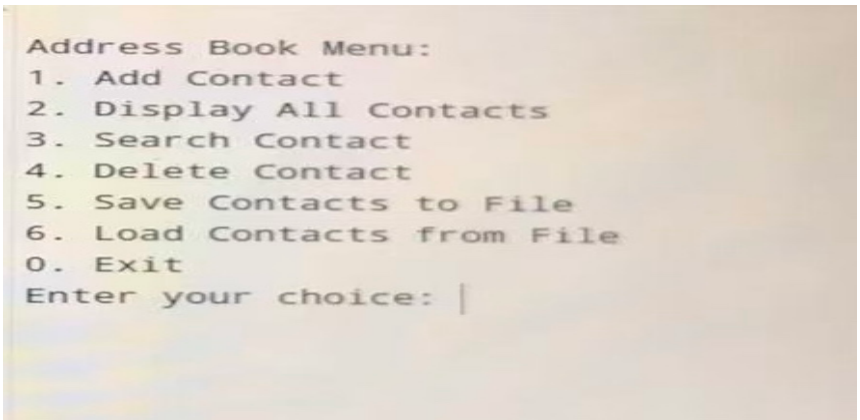


Fig1:Program interface of the system

4.6 Test Results

To test the functionality of the address book system, a sample record was used.

The steps below show the test input and the expected behavior of the program.

Test Input

1. Add a new contact with the following information:

- Name: Daniel
- Street: 45 Maple Avenue
- City: Riverdale
- Postcode: 90210
- Email: daniel.morgan@mail.com
- Country: Canada
- State: ON

2. Display the address book.

3. Search for the contact named “Daniel”

4. Delete the record for “Daniel”

Expected Output

- After adding the record, Daniel’s information appears correctly in the displayed address book.

- Searching for “Daniel” successfully retrieves his contact details.
- Deleting Daniel removes his record from the list, and he no longer appears in the address book.

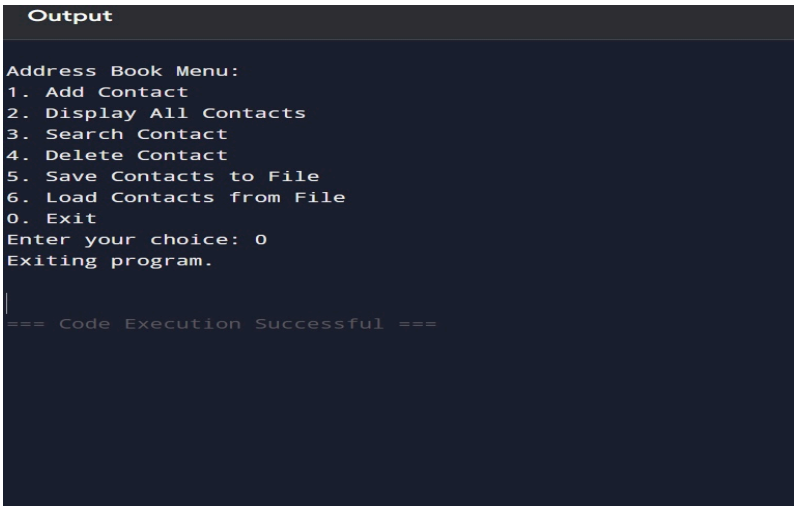


Fig 1: output of the program

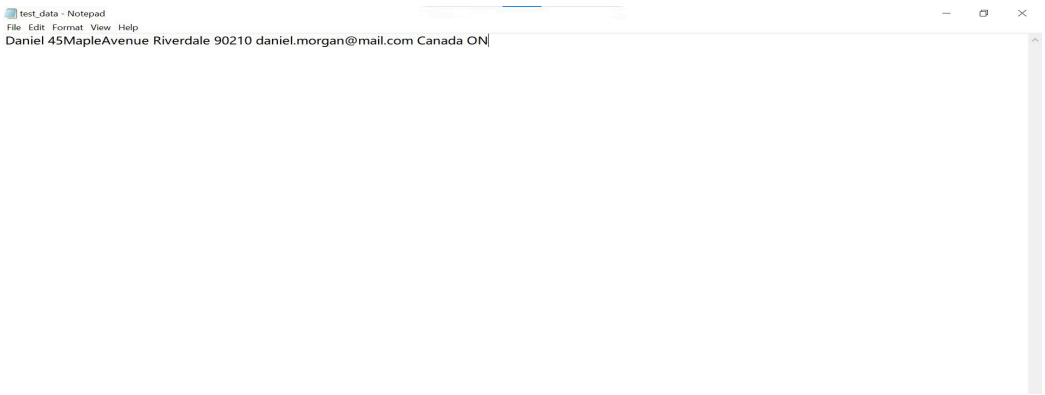


Fig 2:Test data stored in a text file (Notepad)

4.7 Design Experience

This course design project focused on developing a simple address book system using a doubly linked list in the C programming language.

The use of a dynamic data structure made it possible to insert, delete, and

search records efficiently without needing to shift or reorder memory.

Throughout the implementation, several key features were completed, including adding new contacts, displaying all stored information, searching for a specific name, deleting a record, and saving or loading data through a text file.

These functions work together to create a functional and easy-to-use address management tool.

Overall, the project provided practical experience in handling linked list operations, managing dynamic memory, and working with file input/output in C.

The design succeeds in offering a clear structure, efficient data handling, and straightforward user interaction.